PROJECTREPORT

CLI BANK SIMULATOR

VIT BHOPAL

vityarthi

YOUR LEARNING DESTINATION

PROJECT TITLE : CLI BANK SIMULATOR

COURSE TITLE : FUNDAMENTALS IN AI AND ML

COURSE CODE : CSA2001

REGISTRATION NUMBER : 25BAI10093

COURSE TYPE : FLIPPED COURSE

1. PROJECT OVERVIEW

In our daily lives, we use banking apps constantly, but we rarely stop to think about the logic that prevents a balance from disappearing or ensures a withdrawal doesn't exceed the available funds. This project, the **CLI Bank Simulator**, is a practical exploration into the "engine" of a financial application.

Instead of building a fancy website, I focused on creating a lightweight, Command Line Interface (CLI) tool. It handles the core essentials: letting a user open an account, deposit money, and take money out, all while keeping a permanent record on the computer's hard drive.

2. THE PROBLEM & OBJECTIVE

Most beginner programming projects lose all their data as soon as you close the window. For a banking app, that is a failure. My main objectives were:

- **Data Persistence:** I wanted to make sure that if I deposited ₹ 500 today, it would still be there when I opened the program tomorrow.
- **Input Reliability:** In the real world, users make mistakes. I needed a system that wouldn't crash if someone typed "ten" instead of "10."
- **Minimalism:** By using only the Python Standard Library, I aimed to create a tool that is fast, secure, and doesn't require any internet connection or heavy installations.

3. MY METHODOLOGY & TECHNICAL CHOICES

I took a "Security-First" approach to the coding process. I used **Object-Oriented Programming (OOP)** because it allows the bank's data and its actions (like depositing or withdrawing) to live together in one organized "BankSystem" class.

Key Technical Decisions:

- **The Choice of JSON:** I used `json` for storage because it's much more professional than a simple text file. It stores data in a "key-value" format, making it very easy to look up a specific user's balance.
- **The OS Guard:** I used the `os` module to act as a guard. It checks if the database file exists before the program starts, which prevents the "File Not Found" errors that often break student projects.

- **Handling the "Empty File" Bug:** A big challenge was the `JSONDecodeError` which happens if the data file is empty. I wrote a specific check to catch this, ensuring the program starts fresh with an empty dictionary instead of crashing.

4. CORE SYSTEM FEATURES

The project is built around four main "User Stories":

1. **Account Creation:** The system checks if a name is already taken to avoid duplicates and ensures that the opening balance is a positive number.
2. **Checking Balances:** A simple, fast lookup that pulls the latest data from the JSON file.
3. **Deposit Logic:** Updates the balance and immediately saves it to the disk.
4. **Withdrawal Safety:** This is the most critical logic. It performs a "Check-Before-Action" to ensure a user never withdraws more than they own.

5. SYSTEM PERFORMANCE

- **Speed:** Because there are no heavy graphics, the math and file saving happen almost instantly (under 0.1 seconds).
- **Stability:** By using `try...except` blocks, I made the program "Human-Proof." It catches typing errors and gives a polite error message instead of showing scary red code.
- **Portability:** The entire project is just one `.py` file. It can be moved to any laptop and will run perfectly without needing to install anything extra.

6. WHAT I LEARNED FROM THIS PROJECT

Building this was a huge learning curve. It taught me that the most important part of coding isn't the "math"—it's the **logic flow**.

- I learned how to manage files and data structures effectively.
- I realized that "cleaning" user input (like using `.strip().title()`) is essential to make a program feel professional.
- Most importantly, I learned how to debug my own work by anticipating where a user might make a mistake.

7. CONCLUSION

The **CLI Banking System** might look simple on the outside, but the logic inside is robust and reliable. It successfully simulates a real-world bank's ledger and provides a solid foundation for adding more advanced features like passwords, transaction history, or even automated interest calculations in the future.

SCREEN SHOTS OF CODE AND OUTPUT

File Edit Format Run Options Window Help

```python
import json
import os

class BankSystem:
    def __init__(self):
        self.data_file = "bank_data.json"
        self.accounts = self.load_data()

    def load_data(self):

        if os.path.exists(self.data_file):
            try:
                with open(self.data_file, 'r') as f:
                    content = f.read().strip()
                    return json.loads(content) if content else {}
            except (json.JSONDecodeError, ValueError):
                return {}
        return {}

    def save_data(self):
        """Saves current accounts to the JSON file."""
        with open(self.data_file, 'w') as f:
            json.dump(self.accounts, f, indent=4)

    def create_account(self):
        print("---  Open New Account ---")
        name = input("Enter your full name: ").strip().title()
        if name in self.accounts:
            print(" Error: An account with this name already exists.")
            return

        try:
            initial_deposit = float(input("Enter initial deposit amount: ₹"))
            if initial_deposit < 0:
                print(" Error: Initial deposit cannot be negative.")
                return
            self.accounts[name] = initial_deposit
            self.save_data()
            print(f" Success! Account created for {name} with ₹{initial_deposit}")
        except ValueError:
            print(" Error: Invalid amount entered.")

    def check_balance(self):
        name = input("\nEnter your account name: ").strip().title()
        if name in self.accounts:
            print(f"  Current Balance for {name}: ₹{self.accounts[name]}")
        else:
            print("  Error: Account not found.")
```

```python
    def deposit(self):
        name = input("Enter account name for deposit: ").strip().title()
        if name in self.accounts:
            try:
                amount = float(input("Enter amount to deposit: ₹"))
                if amount > 0:
                    self.accounts[name] += amount
                    self.save_data()
                    print(f"Deposited ₹{amount}. New Balance: ₹{self.accounts[name]}")
                else:
                    print(" Error: Amount must be positive.")
            except ValueError:
                print("Error: Invalid amount entered.")
        else:
            print(" Error: Account not found.")

    def withdraw(self):
        name = input("\nEnter account name for withdrawal: ").strip().title()
        if name in self.accounts:
            try:
                amount = float(input("Enter amount to withdraw: ₹"))
                if 0 < amount <= self.accounts[name]:
                    self.accounts[name] -= amount
                    self.save_data()
                    print(f"  Withdrew ₹{amount}. Remaining Balance: ₹{self.accounts[name]}")
                elif amount > self.accounts[name]:
                    print(" Error: Insufficient funds!")
                else:
                    print("  Error: Amount must be positive.")
            except ValueError:
                print(" Error: Invalid amount entered.")
        else:
            print(" Error: Account not found.")
```

```python
def main_menu():
    bank = BankSystem()
    while True:
        print("\n" + "="*35)
        print("    VIT COMMUNITY BANK")
        print("="*35)
        print(" 1. Create New Account")
        print(" 2. Check Balance")
        print(" 3. Deposit Money")
        print(" 4. Withdraw Money")
        print(" 5. Exit")

        choice = input(" Select an option (1-5): ")

        if choice == '1':
            bank.create_account()
        elif choice == '2':
            bank.check_balance()
        elif choice == '3':
            bank.deposit()
        elif choice == '4':
            bank.withdraw()
        elif choice == '5':
            print("Thank you for banking with us. Goodbye!")
            break
        else:
            print(" Invalid choice. Please try again.")

if __name__ == "__main__":
    main_menu()
```

```
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11
Enter "help" below or click "Help" above for more informat

============= RESTART: C:\Users\91978\Desktop\cli bank sim

====================================
    🏦  VIT COMMUNITY BANK  🏦
====================================
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 1

--- 📝 Open New Account ---
Enter your full name: Bavesh
Enter initial deposit amount: ₹5500
 ✅ Success! Account created for Bavesh with ₹5500.0

====================================
    🏦  VIT COMMUNITY BANK  🏦
====================================
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 2

Enter your account name: Bavesh
 💰 Current Balance for Bavesh: ₹5500.0
```

```
==================================
    🏦  VIT COMMUNITY BANK  🏦
==================================
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 2

Enter your account name: Bavesh
 💰 Current Balance for Bavesh: ₹5500.0

==================================
    🏦  VIT COMMUNITY BANK  🏦
==================================
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 4

Enter account name for withdrawal: Bavesh
Enter amount to withdraw: ₹500
 ☑ Withdrew ₹500.0. Remaining Balance: ₹5000.0

==================================
    🏦  VIT COMMUNITY BANK  🏦
==================================
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 2

Enter your account name: Bavesh
 💰 Current Balance for Bavesh: ₹5000.0
```

```
-----------------------------------
 1. Create New Account
 2. Check Balance
 3. Deposit Money
 4. Withdraw Money
 5. Exit

 Select an option (1-5): 2

Enter your account name: Bavesh
 💰 Current Balance for Bavesh: ₹5000.0
```

REFERENCES :

1.VITDYATHI COURSE

2.GOOGLE (DOUBT CLARIFICATION AND DOCUMENTATION)

3.PERR AND MENTOR FEEDBACKS FROM

DR SUBASH CHANDRA BOSE

RAGHUL SK - 25BAI10288

DILIP S – 25BAI10590

GOKUL PRASAD – 25BAI10148

SUTHARSAN - 25BAI10706